

Relatório Técnico — Análise de Logs de Agents CLI

Arquivos analisados:

- `claude-litellm-qwen2.5.log`
- `aider.log`
- `crush-qwen3.5.log`
- `crush-qwen2.5.log`
- `opencode-qwen2.5-01.log`

1. Visão Geral Arquitetural

Agent	Provider	Endpoint	Streaming	Middleware/ Proxy	Característica dominante
Claude Code + LiteLLM	Ollama via LiteLLM	<code>/v1/chat/completions</code>	Sim	LiteLLM Router + Proxy	Arquitetura multi-agent + tool orchestration
Aider	Ollama/ LiteLLM	<code>/api/generate</code>	Sim e não-stream	LiteLLM	Prompt extremamente procedural
Crush	Ollama	<code>/v1/chat/completions</code>	Sim	Direto	Autonomia agressiva
OpenCode	Ollama	<code>/v1/chat/completions</code>	Sim	Direto	CLI minimalista com forte economia de tokens

2. Fluxo Inferido por Agent

2.1 Claude Code + LiteLLM

Arquitetura inferida

Componentes detectados

- LiteLLM Router
- LiteLLM Proxy
- Ollama provider
- Sistema de callbacks assíncronos

- Multi-agent orchestration
- Session pooling HTTP
- Function/tool abstraction
- JSON schema driven tool calls

Evidências

Routing strategy: simple-shuffle

SESSION REUSE: Created shared aiohttp session for connection pooling

Initialized litellm callbacks

provider = ollama

Estratégia de roteamento

O LiteLLM está usando:

Routing strategy: simple-shuffle

Inferência:

- Estratégia de balanceamento simples
- Possibilidade de múltiplos backends/modelos
- Infra preparada para failover horizontal
- Proxy desacoplado do modelo real

Reuso de sessão HTTP

Evidência:

SESSION REUSE: Created shared aiohttp session for connection pooling
limit=1000
limit_per_host=500

Características inferidas:

- Reuso agressivo de conexões
- Otimização de latência
- Redução de overhead TCP/TLS

- Arquitetura preparada para paralelismo alto
-

Sistema de callbacks/hooks

Callbacks detectados:

- deployment_callback_on_success
- SkillsInjectionHook
- VirtualKeyModelMaxBudgetLimiter
- MaxBudgetLimiter
- MaxParallelRequestsHandler_v3
- CacheControlCheck
- ResponsesIDSecurity
- MaxIterationsHandler
- MaxBudgetPerSessionHandler
- ServiceLogging

Inferência arquitetural

Claude Code/LiteLLM implementa:

- middleware pipeline
- budget limiting
- controle de paralelismo
- controle de iterações
- segurança de response IDs
- skill injection
- telemetria
- cache layer

Arquitetura extremamente modular.

Multi-agent orchestration

O prompt contém definição explícita de subagents.

Subagents detectados:

- claude
- claude-code-guide
- Explore
- general-purpose
- Plan
- statusline-setup

Evidências

Launch a new agent to handle complex, multi-step tasks

When you launch multiple agents for independent work

run agents in parallel

foreground vs background

Capacidades inferidas

- execução concorrente
- coordenação multi-agent
- isolamento por worktree
- task delegation
- async orchestration
- background jobs
- resumable agents
- persistent context handoff

Estratégia de contexto

O prompt é extremamente grande.

Características:

- regras hierárquicas
- workflow procedural
- few-shot extensive prompting
- behavioral constraints
- autonomous planning
- editing protocols
- operational safety
- tool governance

Sinais fortes de prompt-engineering avançado

READ BEFORE EDITING

BE AUTONOMOUS

TEST AFTER CHANGES

USE EXACT MATCHES

LOAD MATCHING SKILLS

Inferência

Claude Code usa:

- prompt como runtime policy engine
 - behavioral simulation
 - state machine implícita
 - pseudo-operating-system prompt
 - workflow compiler via prompting
-

Tool abstraction via JSON schema

Ferramentas definidas via JSON Schema:

Agent Tool

Contém:

- description
- prompt
- subagent_type
- model override
- isolation
- background execution

AskUserQuestion Tool

Contém:

- multiple choice schema
- annotations
- metadata
- previews
- multi-select

Inferência

Claude Code converte ferramentas em:

- declarative interfaces

- schema-driven execution
 - UI-native orchestration
 - structured interaction runtime
-

Uso de contexto e compressão

Padrões detectados:

- instruções extremamente comprimidas semanticamente
- reutilização de regras macro
- tool abstraction em vez de instruções repetidas
- compressão operacional via protocolos

Exemplo:

```
Trust but verify
```

Uma única frase substitui múltiplos comportamentos.

Tamanho aproximado do prompt

Evidências:

```
num_predict: 32000
```

Além disso:

- dezenas de páginas de system prompt
- múltiplas definições de agents
- exemplos few-shot extensos
- protocolos operacionais

Inferência:

- prompt total muito grande
 - forte consumo de contexto
 - arquitetura dependente de long-context
-

Erros detectados

Incompatibilidade com reasoning/thinking

Erro:

```
APIConnectionError: OllamaException - {"error":"\\"qwen2.5-tuned:latest\\" does not support thinking"}
```

Inferência

Claude Code esperava:

- modelo com reasoning mode
- hidden chain-of-thought
- ou API compatível com thinking tokens

Qwen via Ollama não suportava essa feature.

Fallback behavior

Sinais detectados:

```
ageneric_api_call_with_fallbacks
```

Indica:

- retry/fallback orchestration
 - abstração resiliente de providers
 - pipeline tolerante a falhas
-

Latência

Sem métricas completas.

Mas:

- session pooling reduz overhead
 - callbacks aumentam overhead interno
 - prompts enormes aumentam TTFT
 - orchestration multi-agent aumenta latência acumulada
-

2.2 Aider

Arquitetura inferida

Características principais

- Prompt procedural rígido

- SEARCH/REPLACE editing model
 - Strong deterministic editing
 - File-scoped patching
 - Conversational editing
-

Endpoint e provider

```
POST /api/generate
```

```
agent_header: litellm/1.81.10
```

```
model: qwen2.5-tuned:latest
```

Inferência

- Aider está usando LiteLLM como camada de compatibilidade
 - backend real = Ollama/Qwen
 - interface desacoplada do provider
-

Estratégia de edição

Aider usa:

```
SEARCH/REPLACE block
```

Características

- edição determinística
 - patching textual
 - baixa ambiguidade
 - alta compatibilidade com LLMs menores
 - reduz hallucination estrutural
-

Controle rígido de formato

Padrões:

```
ONLY EVER RETURN CODE IN A SEARCH/REPLACE BLOCK
```

EXACTLY MATCH

FULL file path

Inferência

Aider tenta:

- transformar geração em transformação textual formal
- minimizar parsing ambíguo
- reduzir erro de patch application
- evitar drift estrutural

Estratégia de contexto

O prompt inclui:

- regras operacionais
- exemplos extensos
- templates fixos
- constraints de shell
- constraints de output
- workflow guiado

Mas diferente do Claude Code:

Claude Code	Aider
agentic runtime	deterministic editor
orchestration	patch generation
autonomous	guided editing
tool ecosystem	text transformation

Padrão de reasoning

Aider força reasoning explícito:

Think step-by-step

Mas imediatamente restringe saída:

ONLY EVER RETURN CODE

Inferência

Reasoning provavelmente:

- interno
- descartado antes da resposta
- usado apenas para construir patch

Manipulação de arquivos

Características:

- edição somente de arquivos compartilhados
- file boundary enforcement
- edição incremental
- criação explícita de arquivos
- rename via shell

Evidências

Only create SEARCH/REPLACE blocks for files that the user has added

Uso de shell

Aider instrui:

suggest shell commands

Mas não executa.

Diferença importante:

Sistema	Execução real
Claude Code	tool execution
Crush	autonomous execution
Aider	shell suggestion

Latência

Evidência:

```
"latency_ms":48550
```

~48.5 segundos.

Possíveis causas

- prompt extremamente longo
 - contexto enorme
 - modelo local Qwen
 - stream generation longa
 - JSON-heavy output
-

Prompt size

Evidência:

```
"prompt_tokens_est":2698
```

Muito alto para:

- apenas uma task simples
- CRUD API generation

Indica:

- enorme overhead operacional
 - prompt system dominante
 - baixa eficiência contextual
-

Fluxo inferido

```
User request
→ gigantic system prompt
→ editing protocol injection
→ structured patch generation
→ shell suggestion
→ optional commit message generation
```

Commit message generation

Existe pipeline separado:

You are an expert software engineer that generates concise, one-line Git commit messages

Inferência

Aider usa:

- task-specialized prompts
- micro-agents implícitos
- specialized inference stages

2.3 Crush

Arquivos:

- `crush-qwen2.5.log`
- `crush-qwen3.5.log`

Arquitetura inferida

Crush implementa:

- highly autonomous agent runtime
- CLI-native orchestration
- aggressive execution model
- continuous workflow loop
- tool-driven execution

Filosofia operacional

Evidências

BE AUTONOMOUS

Don't ask questions

Systematically try alternative strategies

Keep going until query is completely resolved

Inferência

Crush tenta maximizar:

- autonomy
- task completion rate
- uninterrupted execution
- agent persistence
- self-directed exploration

Muito mais agressivo que Aider.

Workflow interno

Fluxo detectado:

```
search codebase
→ read files
→ identify changes
→ edit
→ run tests
→ fix failures
→ continue until done
```

Características

- explicit execution loop
 - retry mentality
 - self-healing workflow
 - iterative execution
-

Estratégia de edição

Diferente do Aider.

Crush possui ferramentas explícitas:

- edit
- multiedit
- write

Evidências

Never use apply_patch

Use edit or multiedit instead

Inferência

Crush provavelmente:

- opera sobre AST/text hybrid tooling
- possui executor de patches interno
- mantém runtime state

Estratégia de contexto

Prompt enorme semelhante ao Claude Code.

Mas diferenças importantes:

Claude Code	Crush
Multi-agent orchestration	Single-agent autonomy
Tool schemas	Workflow rules
Delegation	Persistent execution
Structured orchestration	Aggressive completion loops

Context governance

Padrões:

READ BEFORE EDITING

USE EXACT MATCHES

TEST AFTER CHANGES

Objetivo

Evitar:

- broken edits
 - hallucinated patches
 - formatting corruption
 - stale context
-

Retry strategy

Evidências

If stuck, try different approach

don't repeat failures

Systematically try alternative strategies

Inferência

Crush implementa:

- implicit retry orchestration
 - self-corrective behavior
 - branch-based exploration
 - adaptive execution
-

Segurança operacional

SECURITY FIRST

Only assist with defensive security tasks

Inferência

- política embutida no runtime
 - hard safety constraints no prompt
 - não depende apenas do modelo base
-

Controle de output

Under 4 lines

No preamble

No postamble

Objetivo

- reduzir token waste
- melhorar UX CLI
- reduzir context pollution

Manipulação de arquivos

Características:

- leitura obrigatória antes de edição
- whitespace-sensitive editing
- contextual matching
- exact replacement
- incremental changes

Padrões de reasoning

O reasoning é procedural e operacional.

Não há:

- discussões conceituais
- explicações longas
- chain-of-thought explícito

Há:

- action-oriented planning
 - workflow execution
 - self-verification
 - execution loops
-

Diferença entre Qwen2.5 e Qwen3

Os prompts são praticamente equivalentes.

Diferença principal:

```
model=qwen3-tuned:latest
```

vs

```
model=qwen2.5-tuned:latest
```

Não há diferença arquitetural detectável nos logs.

2.4 OpenCode

Arquitetura inferida

OpenCode aparenta ser:

- CLI minimalista
 - tool-oriented runtime
 - strongly optimized for token efficiency
 - compact operational prompting
-

Estratégia dual-prompt

Foram detectados dois prompts distintos.

Prompt 1

Thread title generator.

Prompt 2

Main coding assistant.

Inferência

OpenCode separa:

- UX metadata tasks
- coding tasks

Pipeline modular.

Token optimization

Padrões recorrentes:

- Keep your responses short
- One word answers are best
- minimize output tokens

Inferência

OpenCode é fortemente otimizado para:

- CLI responsiveness
 - low token usage
 - fast inference
 - reduced context growth
-

Proatividade controlada

Diferente do Crush.

Evidência

You are allowed to be proactive, but only when the user asks

Comparação

Agent	Estratégia
Crush	máxima autonomia
OpenCode	autonomia controlada
Aider	execução guiada
Claude Code	orchestration complexa

Ferramentas

OpenCode menciona:

- WebFetch
- search tools
- Bash
- file editing
- concurrent tool calls

Mas o prompt é muito mais compacto.

Estratégia de contexto

OpenCode usa:

- instruções compactas
- poucos exemplos
- alta compressão semântica
- forte reutilização conceitual

Muito mais eficiente contextualmente que Aider.

Prompt size

Evidência:

```
"prompt_tokens_est": 746
```

Muito menor que:

- Aider (~2698)
 - Claude Code (aparentemente enorme)
-

Latência

Evidência:

```
"latency_ms": 1860
```

~1.8 segundos.

Extremamente menor que Aider.

3. Comparação Técnica Global

3.1 Filosofia arquitetural

Sistema	Filosofia
Claude Code	Orquestrador multi-agent
Aider	Editor determinístico
Crush	Executor autônomo
OpenCode	CLI minimalista eficiente

3.2 Estratégia de prompting

Sistema	Estratégia
Claude Code	Runtime operacional completo
Aider	Protocolos rígidos de edição
Crush	Workflow procedural autônomo
OpenCode	Compressão extrema

3.3 Uso de contexto

Sistema	Uso de contexto
Claude Code	Muito alto
Aider	Alto
Crush	Alto
OpenCode	Moderado/baixo

3.4 Tool orchestration

Sistema	Orchestration
Claude Code	Multi-agent + schemas
Crush	Workflow tools
OpenCode	Tool calls concorrentes

Sistema	Orchestration
Aider	Quase sem execução real

3.5 Estratégia de reasoning

Sistema	Tipo
Claude Code	Hierárquico/orquestrado
Aider	Determinístico guiado
Crush	Operacional iterativo
OpenCode	Compactado e pragmático

3.6 Estratégia de segurança

Sistema	Segurança
Claude Code	Middleware + policies
Crush	Hard rules no prompt
OpenCode	Behavioral constraints
Aider	Estrutura de edição restrita

3.7 Latência observada

Sistema	Latência
OpenCode	~1.8s
Aider	~48.5s
Claude Code	Não conclusiva, mas alta complexidade
Crush	Não mensurada

4. Padrões Globais Detectados

Todos os agents usam

- prompting como runtime

- behavioral governance
- deterministic constraints
- anti-hallucination heuristics
- context engineering
- tool-aware prompting
- CLI optimization
- exact-edit strategies

5. Principais Diferenças Filosóficas

Sistema	Objetivo principal
Claude Code	Delegar trabalho para subagents
Aider	Gerar patches seguros
Crush	Completar tarefas autonomamente
OpenCode	Maximizar velocidade e eficiência

6. Conclusões Técnicas Inferidas dos Logs

Claude Code

- arquitetura mais sofisticada
- maior overhead operacional
- maior capacidade de orchestration
- runtime mais complexo
- dependência forte de contexto longo

Aider

- extremamente determinístico
- workflow textual rígido
- menor autonomia
- foco em confiabilidade de patch

Crush

- maior autonomia operacional
- mindset de execução contínua
- runtime agressivo
- orientação forte a task completion

OpenCode

- maior eficiência contextual

- menor overhead
- menor latência
- prompting muito mais comprimido
- UX focada em velocidade